

Lecture 01 — Introduction

Prof. Indranil Sengupta · Department of Computer Science and Engineering · IIT Kharagpur · expanded for the Chip Design Associate programme

- Theory 1 · Why this course exists — the six objectives, and what a hardware description language actually is
- Theory 2 · The problem — complexity, Moore's Law, and why manual design was ruled out
- Theory 3 · The answer — a standardized design flow, and the CAD tools that walk it
- Theory 4 · The steps, one at a time — behavioural, data path, logic, physical, and the rest
- Practical · Five labs · install the toolchain, then check every claim in the lecture on your own machine



Where This Lecture Sits

Prof. Indranil Sengupta · Dept. of Computer Science and Engineering · IIT Kharagpur (NPTEL)

Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Introduction Design representation Verilog basics 5 lectures	Language features Operators Modeling examples 6 lectures	Description styles Procedural assignment 4 lectures	Blocking and non-blocking User-defined primitives 5 lectures	Test benches Finite state machines 4 lectures	Synthesizable Verilog · Recommended practices 5 lectures	Memory · Register banks · Pipelining Switch level 7 lectures	Pipelined processor — a full case study 5 lectures



YOU ARE HERE Lecture 01 · Introduction — the VLSI design flow, and why a hardware description language is the thing that makes it possible

Eight weeks, forty-one lectures

Lecture 01 carries no Verilog syntax at all. Its job is to give you the map: what a VLSI design flow is, what each step of it produces, and why a language like Verilog has to exist at all. Everything in Weeks 2–8 is a detail of this picture.



Terminal Outcomes

Chip Design Associate · Module 2 · NOS NIE/ELE/N0102 — Verilog RTL Coding for Synthesis

TO-1

Explain the design cycle of a VLSI circuit — from a design idea down to a fabricated chip or a programmed FPGA.

Fully served. This lecture IS the design cycle.

TO-2

Describe Verilog syntax, the levels of abstraction in Verilog programming, and testbench simulation.

Levels of abstraction: served. Syntax and testbenches: opened.

TO-3

Design and develop IPs using Verilog.

Groundwork only — the vocabulary every later lecture uses.

TO-4

Emulate, debug and characterise reusable IPs.

Groundwork only — introduces simulation and verification.

Outcome 1 is this lecture, almost word for word

"Explain the design cycle of a VLSI circuit." That is the whole of Lecture 01. The remaining outcomes are opened here and closed over the rest of the module.



Learning Outcomes

Each one is assessed by something you produce or run, not by something you recite.

- 1 Name the five levels of the VLSI design flow in order, and say what each level's description is made of.
- 2 Explain what a CAD tool actually does: it reads an HDL description and writes a more detailed HDL description.
- 3 Distinguish a behavioural description from a structural one, and give an example of each for the same circuit.
- 4 Define a netlist as a directed graph, and read one at the functional, gate and transistor level.
- 5 State Moore's Law, and explain why it is an economic observation rather than a law of physics.
- 6 Install and verify a complete open-source Verilog toolchain: Icarus Verilog, GTKWave, Yosys and Verilator.
- 7 Run a synthesis tool on a small design and read the cell count it reports at each level of the flow.
- 8 Explain the ASIC / FPGA trade-off in terms of speed, flexibility and cost.

Outcomes 1-5 are theory. Outcomes 6-8 are the practical component, and are assessed at the keyboard.



The Original Lecture, and How This Session Expands It

Every slide of Lecture 01 is kept, in the lecturer's own order. The right-hand column is what has been added around it.

SLIDE TIME	ORIGINAL SLIDE TITLE	WHAT THIS SESSION ADDS
00:58	Main Objectives of the Course	What an HDL is and is not; a concurrency example
03:06	VLSI Design Process	The complexity arithmetic; the area/speed/power triangle
07:25	First planar IC vs. a modern processor	The actual transistor counts, and what they mean
08:23	Moore's Law	Real data to 2022; why it is economics, not physics
11:30	CMOS · FinFET · Quantum?	What a 'nanometre' means; where the node numbers stand now
13:19	VLSI Design Flow	Each step named, with its input and its output
14:40	Need to use CAD tools	The central idea: a CAD tool is an HDL→HDL transformer
17:16	Two Competing HDLs	The same counter written in Verilog and in VHDL
18:23	Simplistic View of Design Flow	One circuit carried down all five levels
19:39	Behavioral design	The four ways of specifying behaviour, each illustrated
20:54	Data path design	RTL components; what 'register transfer' means
21:20	Netlist as a graph (whiteboard)	The graph drawn properly, at three levels of abstraction
22:46	Logic design	What a standard cell library actually contains
24:27	Physical design and Manufacturing	The layers; and the ASIC / FPGA trade-off
25:52	Other Steps in the Design Flow	Simulation, formal verification, DFT — and where each returns



HOW TO USE THIS DECK

Theory, Then Proof

THEORY

Four sections that follow the lecturer's own running order, slide by slide. Nothing has been reordered; material has only been added around what is already there.

PRACTICAL

Five labs. Every one of them is a command you run, and every number quoted in this deck came out of running them. If a slide states a figure, a lab produced it.

WORKBOOK

A companion document carries the full tutorial, the installation walkthrough, worked solutions and the exercises. The deck is for the room; the workbook is for afterwards.

The notional slot for Lecture 01 is about half an hour. This deck is deliberately longer: lecture from the section openers to fit the slot, and let the rest carry the labs and the self-study.

THEORY 1

Why This Course Exists

The lecturer opens with six objectives. They are worth reading slowly, because between them they describe everything the next eight weeks will do.

- The six objectives of the course, and what each one will mean
- What a hardware description language is
- And — more important — what it is not



Main Objectives of the Course

Hardware Modeling Using Verilog — the lecturer's own six objectives, with what each one will mean in practice

1

Learn about the Verilog hardware description language.

Syntax, data types, operators, and the module as the unit of design.

2

Understand the difference between behavioral and structural design styles.

Say WHAT the circuit does, or name WHICH PARTS it is built from. This distinction runs through the course.

3

Learn to write test benches and analyze simulation results.

A design you cannot test is a design you cannot trust. Weeks 5 onwards are built on this.

4

Learn to model combinational and sequential circuits.

Logic with no memory, and logic with memory. Almost every real block is a mixture of the two.

5

Distinguish between good and bad coding practices.

Verilog will let you write something that simulates correctly and synthesises into the wrong hardware.

6

Case studies with some complex designs.

Week 8 builds a pipelined processor — datapath and control path — entirely in Verilog.



A Hardware Description Language Is Not a Programming Language

Both are text you type. What the text MEANS is completely different — and this is the single most common beginner error.

C — A PROGRAM

```
a = 1;  
b = 2;  
c = 3;
```

These happen ONE AFTER ANOTHER.

The order of the lines is the order of events. Swap two of them and you change what the program does.

VERILOG — A DESCRIPTION

```
assign a = 1;  
assign b = 2;  
assign c = 3;
```

These are all true AT THE SAME TIME.

Each line describes a piece of wire that exists permanently. Swap two of them and nothing changes at all.

In the Verilog pane there is no "first" and no "next" — only what is connected to what.

You are not writing instructions for a machine to follow.

You are writing a description of a machine that will be built.

The single most common beginner error

Reading Verilog as if it were C. It is not a list of steps to carry out; it is a description of something that exists all at once and stays there. Hold on to that and most of the confusion in Weeks 2 and 3 never happens.



The Same Three Lines, Side by Side

In C, this is a sequence

Three assignments, carried out one after another. After the first line runs, a is 1. Time passes between the lines.

In Verilog, this is a circuit

Three pieces of wire, each permanently driven. There is no "after". All three statements are true at every instant the circuit is powered.

the text looks similar; the meaning is not

<code>// C</code>	<code>// Verilog</code>
<code>a = 1;</code>	<code>assign a = 1;</code>
<code>b = 2;</code>	<code>assign b = 2;</code>
<code>c = 3;</code>	<code>assign c = 3;</code>
<code>// swap two lines and</code>	<code>// swap two lines and</code>
<code>// the program changes</code>	<code>// NOTHING changes</code>

Hardware is not executed. It is built, and then it sits there behaving.

THEORY 2

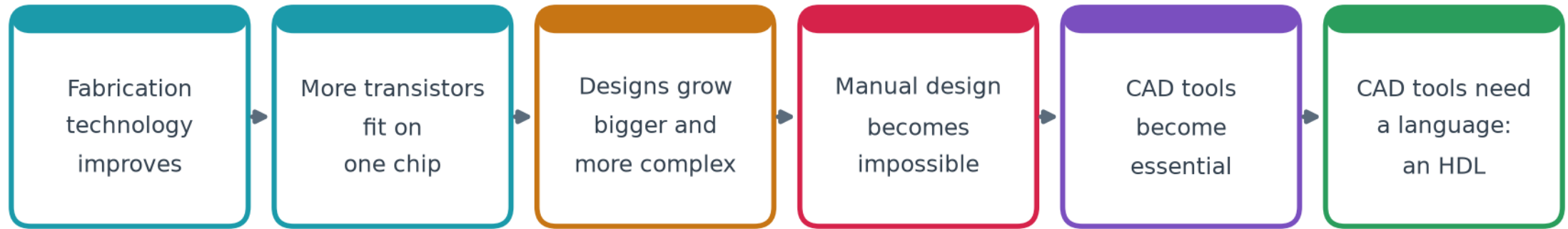
The Problem

Before the flow and the tools and the language, the reason for all three: designs got far bigger, far faster, than anyone could follow by hand.

- The complexity of VLSI circuits, and what drove it
- Moore's Law — and why it is an observation, not a law
- The requirements that conflict: area, speed, power
- Why manual design was ruled out, in arithmetic



VLSI Design Process



And the requirements conflict with each other

Smaller area, higher speed, lower energy — you cannot have all three. Push on one and at least one of the others gets worse. That is why the design flow is not a recipe to follow but a series of engineering choices to make.

Read the chain left to right

Each box is caused by the one before it. The last box — "CAD tools need a language" — is where Verilog enters, and it is the only reason this course has a subject.



The Scale Problem, in Numbers

The two chips the lecturer puts side by side — and the factor between them

FIRST COMMERCIAL PLANAR IC

Fairchild Micrologic
flip-flop, 1961

about 4
transistors

A MODERN PROCESSOR

Intel Core i7 (Nehalem)
quad-core, 2008

731 000 000
transistors

TODAY, FOR COMPARISON

Apple M1 Ultra,
2022

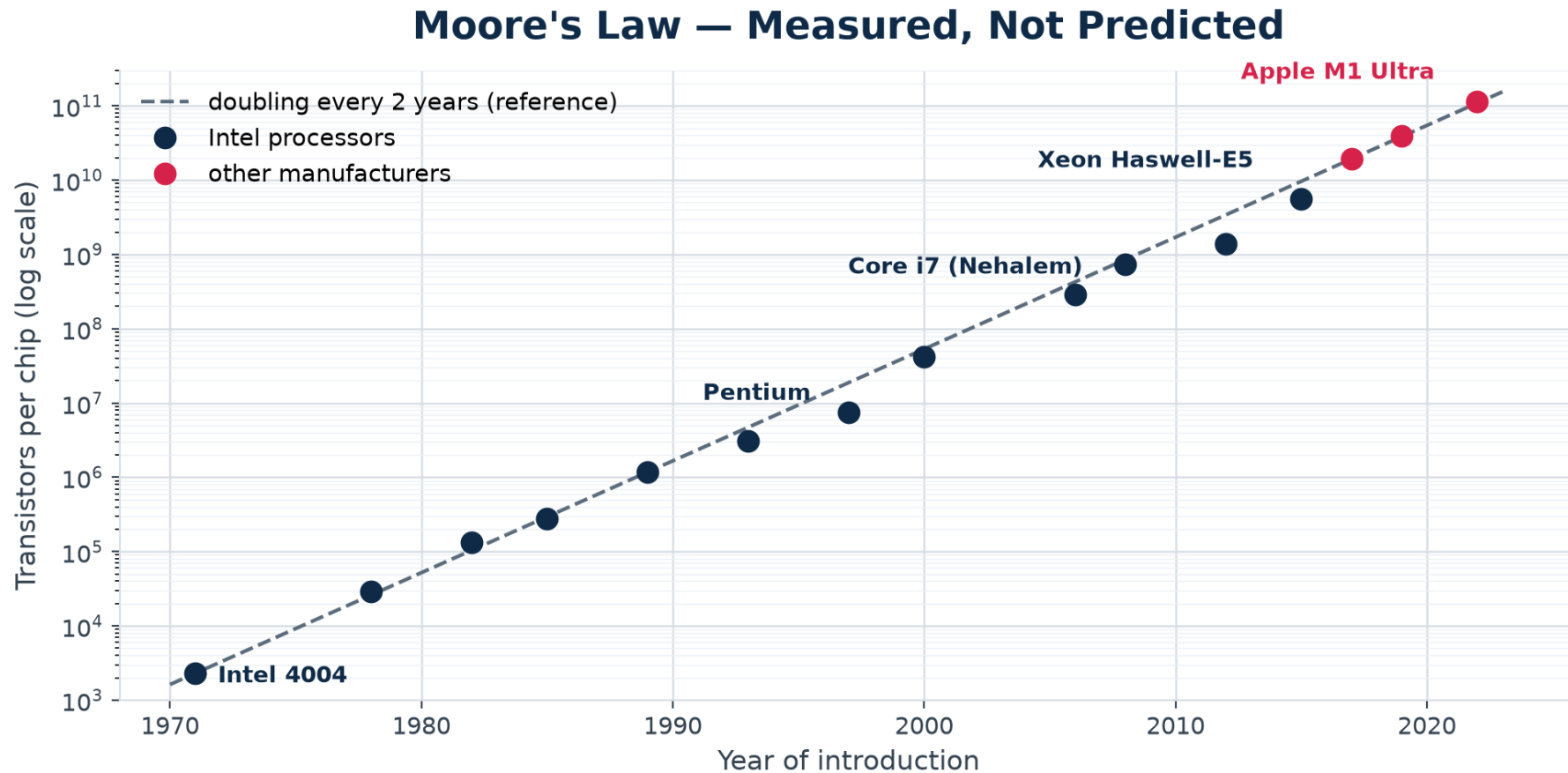
114 000 000 000
transistors

What that factor actually means for a designer

From 4 devices to 731 million is a factor of about 180 million. If you could place and wire one transistor per second, by hand, without ever sleeping, the Nehalem alone would take you about 23 years.



Moore's Law — Measured, Not Predicted



What a straight line on a log scale means

Constant exponential growth. Every point is a real, published transistor count, and the dashed line is a plain doubling every two years anchored on the 4004 — drawn to show that the data really does track it over five decades.



Why It Is an Observation, Not a Law of Physics

WHAT MOORE ACTUALLY SAID

In 1965 Gordon Moore looked at four data points and saw components per chip doubling every year. In 1975 he revised it to every two years. It was a note on what the industry had been doing.

WHY IT KEPT COMING TRUE

Because it became a target. Equipment makers, materials suppliers and chip companies all planned around it, so the investment arrived on schedule. A self-fulfilling forecast.

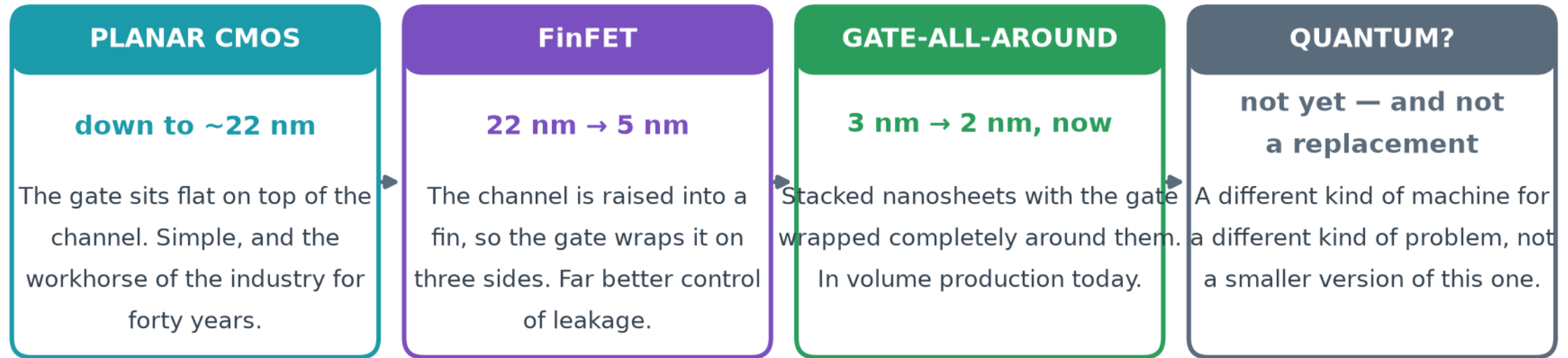
WHY IT IS SLOWING

Transistors are now a few tens of atoms across. Leakage, heat and the cost of each new fab rise faster than the gain. The answer has shifted to chiplets, 3-D stacking and accelerators.

For this course the point is simply this: the number of things on a chip grew far faster than any human's ability to place them. Everything that follows — the flow, the tools, the language — exists to close that gap.



What Made It Possible — and Where It Stands Now



A note on the numbers in the original slide

The lecture says "CMOS up to 22 nm, FinFET 14 nm" — correct when it was recorded. Those node names are now marketing labels rather than measurements: no feature on a "3 nm" chip is 3 nm across. What has not changed at all is the design flow.



What a "Nanometre" Actually Is

A sense of scale, so the node numbers mean something physical

A human hair	about 70 000 nm wide	
A red blood cell	about 7 000 nm across	
Visible light	400 – 700 nm wavelength	
A 22 nm transistor gate	the lecture's "state of the art"	
A silicon atom	about 0.2 nm across	

(bar lengths are illustrative — the real range spans five orders of magnitude)

So a 22 nm gate is roughly 100 silicon atoms across

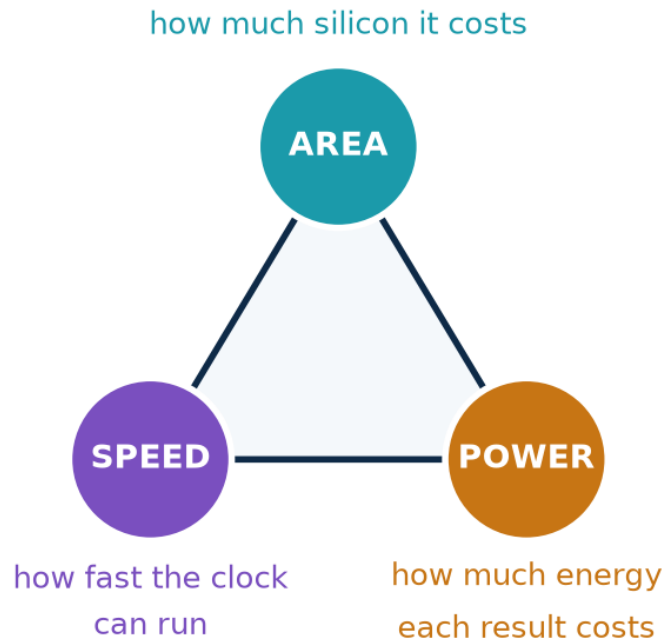
At that size a handful of stray atoms changes a transistor's behaviour. That is why fabrication is statistical, why every chip is tested individually, and why the flow verifies at every level.



2.5 · THE TRADE-OFF

The Three Requirements That Fight Each Other

"Often these requirements are conflicting. If you try to optimize one you may make the other one worse."



Want less **AREA**?

Share one adder between several operations — so the job now takes more clock cycles. Speed falls.

Want more **SPEED**?

Add pipeline registers, or duplicate logic so it runs in parallel. Area rises, and so does switching power.

Want less **POWER**?

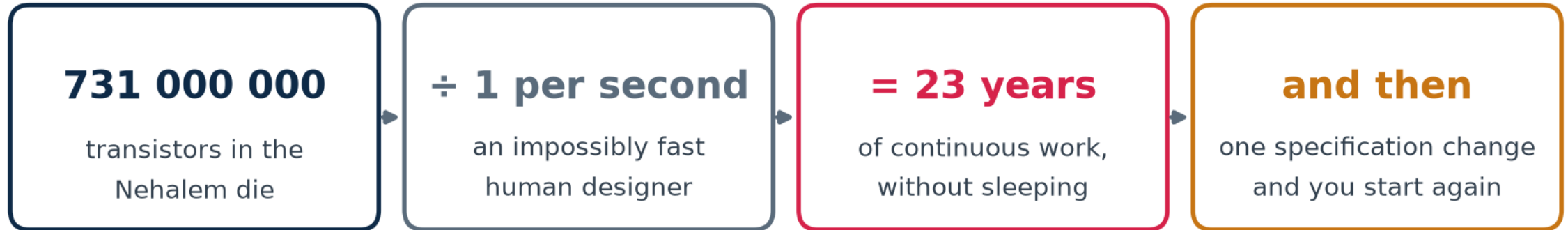
Lower the supply voltage, or gate the clock. Gates get slower, and the control logic gets bigger.

This is why the flow has choices in it

If one design were simply best, a tool could find it and the job would be done. Because these three pull against each other, somebody has to decide which matters most — and that somebody is the designer, not the tool.



"Manual Design Is Simply Ruled Out"



So the job changed

A VLSI designer does not place transistors. A designer writes a DESCRIPTION of what the circuit must do, and a program turns that description into transistors.

**That program is a CAD tool. The description is written in a hardware description language.
That is the whole reason this course exists.**

THEORY 3

The Answer: a Standardized Design Flow

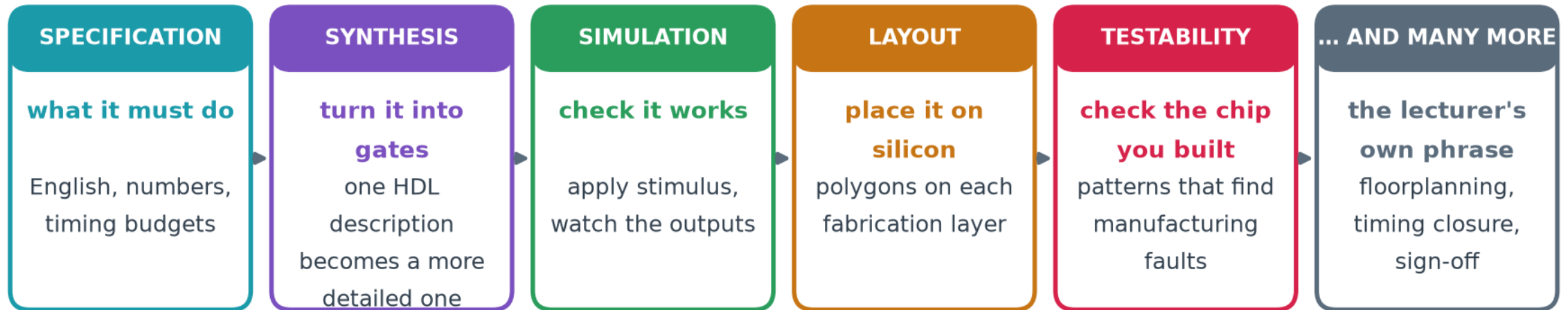
If no one person can place the transistors, the work has to be split into steps with defined inputs and outputs — and handed to programs.

- The steps of the flow, and what each produces
- What a CAD tool actually does: HDL in, more detailed HDL out
- The two competing HDLs, and the others
- One circuit carried down all five levels



VLSI Design Flow

"Step by step, starting from a given specification, down to the actual hardware circuit."



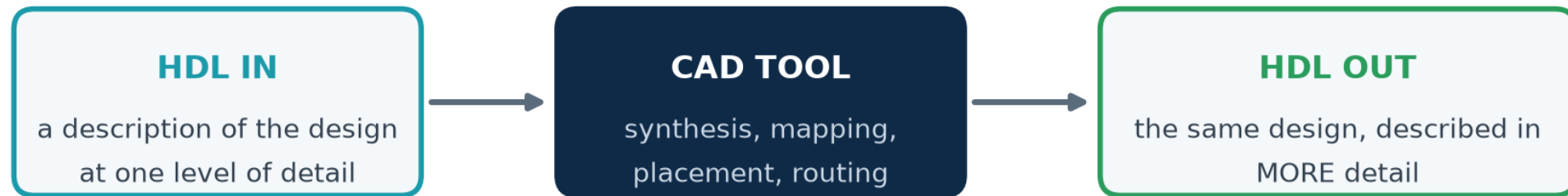
Why it had to be standardized

Dozens of engineers, several companies and a fabrication plant all work on one chip. A standard flow gives each step a defined input, a defined output and a defined check — so the work can be divided up, and a mistake is caught at the step that made it rather than three steps later.



What a CAD Tool Actually Does

"A CAD tool transforms its HDL input into an HDL output that contains more detailed information about the hardware."



And the tool is applied again and again, one level at a time



The idea to hold on to

At every one of these steps, both the input and the output are hardware descriptions. Nothing is ever "compiled into a program" — the design simply gets described in progressively more concrete terms, until the last description is a set of shapes a factory can print.



Two Competing HDLs

VERILOG

C-like syntax, terse, and the language of this course. Dominant in industry in the Americas and much of Asia. Standardised as IEEE 1364.

VHDL

Ada-like syntax, verbose and strongly typed. Strong in Europe, aerospace and defence. Standardised as IEEE 1076.

SystemVerilog

A large superset of Verilog, adding features aimed mostly at verification: classes, constrained random stimulus, assertions. IEEE 1800.

SystemC

A C++ class library rather than a separate language, used for modelling whole systems before the hardware is written.

"Designs are created typically using HDLs, which get transformed from one level of abstraction to the next as the design flow progresses."

Both Verilog and VHDL can describe the same hardware, and a synthesis tool will produce the same gates from either. The choice is a matter of house style, existing code and local convention — not of what the silicon can do. In Lab 5 you will write the same counter in both and compare the waveforms.



The Same Counter, in Both Languages

VERILOG · rtl/counter4.v

Terse. The register, the reset and the increment fit in four lines.

```
module counter4 (  
    input wire      clk,  
    input wire      rst,  
    output reg [3:0] q  
);  
    always @(posedge clk) begin  
        if (rst) q <= 4'd0;  
        else    q <= q + 4'd1;  
    end  
endmodule
```

VHDL · vhdl/counter4.vhd

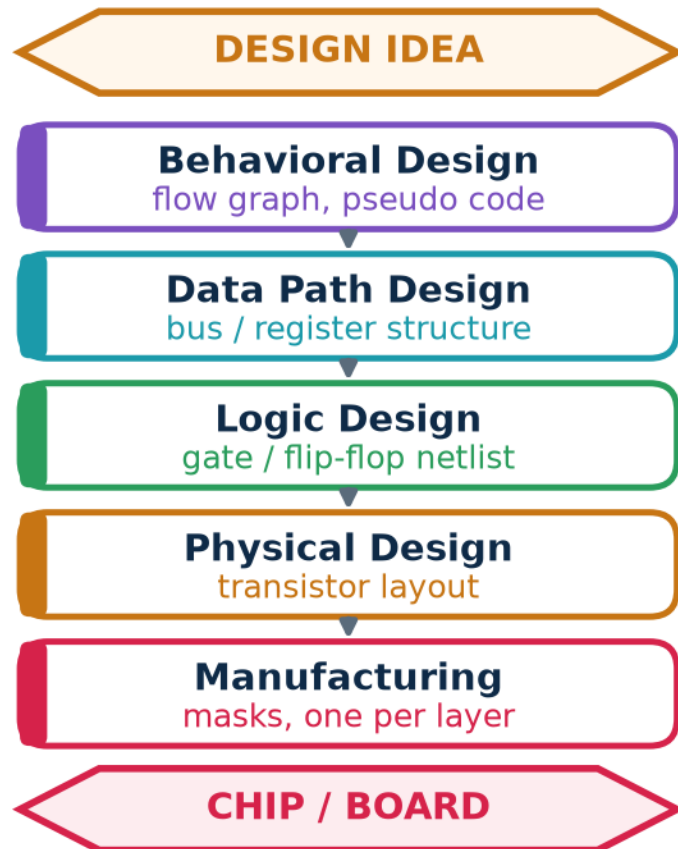
Verbose and strongly typed — the count has to be converted before it leaves the entity.

```
entity counter4 is  
    port (clk : in  std_logic;  
          rst : in  std_logic;  
          q   : out std_logic_vector(3 downto 0));  
end entity;  
architecture rtl of counter4 is  
    signal cnt : unsigned(3 downto 0);  
begin  
    process (clk) begin  
        if rising_edge(clk) then  
            if rst = '1' then cnt <= (others => '0');  
            else                cnt <= cnt + 1;  
            end if;  
        end if;  
    end process;  
    q <= std_logic_vector(cnt);  
end architecture;
```



Simplistic View of the Design Flow

The lecture's own diagram — each box is a description, each arrow is a CAD tool



Each box is a **DESCRIPTION**

Not a stage of manufacture — a document that says what the circuit is, in a different vocabulary each time.

Each arrow is a **TOOL**

A program reads the description above it and writes the one below, adding detail that was not there before.

Detail only ever increases

You never climb back up. Each level fixes decisions that the level above deliberately left open.

This course lives at the top two

Behavioral and data path — exactly where Verilog is written. The rest is done for you by tools.



One Circuit, All Five Levels

A 4-bit counter — the same hardware, described five ways

1 · BEHAVIORAL

"On every rising clock edge, unless reset is asserted, the output becomes one more than it was." A sentence — or a few lines of Verilog.

2 · DATA PATH (RTL)

A 4-bit register, a 4-bit incrementer, and a multiplexer choosing between the incremented value and zero. Named blocks, wired together.

3 · LOGIC

Four D flip-flops and about a dozen gates — AND, XOR, inverters — taken from a standard cell library and connected.

4 · PHYSICAL

Each cell becomes a rectangle of silicon at a fixed position, and every connection becomes a metal track on a specific layer.

5 · MANUFACTURE

A set of masks: one layer of polygons per fabrication step, sent to the foundry.

The point of the exercise

All five describe exactly the same counter. A designer writes the first one; tools produce the other four. In Lab 3 you will run that descent yourself and count what comes out at each level.



What Counts as a "Component" at Each Level

Level	A component is...	A connection is...	Typical count for a small design
Behavioral	a statement	a variable name	tens of lines
Data path (RTL)	register, adder, mux	a named bus or wire	tens of blocks
Logic	gate or flip-flop	a net	hundreds of cells
Physical	a placed cell	a metal track	thousands of shapes
Manufacturing	a polygon	a layer	millions of shapes

Read the last column

The number of things to keep track of grows by roughly a factor of ten at every level. That growth is exactly why a designer works at the top of the table and lets tools handle the bottom.

THEORY 4

The Steps, One at a Time

The lecturer now walks down the ladder, naming each level and saying what its description is made of. This section follows him rung by rung.

- Behavioral design — four ways of saying what a circuit does
- Data path design, and what a netlist really is
- Logic design, standard cells, and the optimisation trade-off
- Physical design, manufacturing, and the FPGA alternative
- The other steps: simulation, formal verification, test



The Four Ways of Saying What a Circuit Does

BOOLEAN EXPRESSION

$$y = a \cdot b + a' \cdot c$$

Algebra over 0 and 1.

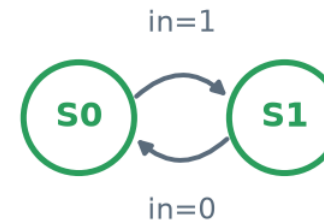
Compact, and exactly what a two-level logic minimiser wants as input.

TRUTH TABLE

a	b	y
0	0	0
0	1	1
1	0	1
1	1	0

Every input combination listed with its output.
Unambiguous, but doubles in size per extra input.

FINITE-STATE MACHINE



States and the transitions between them. The natural way to say what a sequential circuit does.

HIGH-LEVEL ALGORITHM

```
count = 0
repeat forever:
    wait for clock
    count = count + 1
```

An algorithm, written much as you would in C. The highest level of abstraction in the flow.

What they have in common

Not one of these says how the circuit is BUILT. Each one says only what the circuit must DO. That is what makes them behavioural — and it is why a synthesis tool is free to build them in whichever way is cheapest.



Register Transfer Level

- The components are no longer statements. They are **registers, adders, multipliers, multiplexers, decoders and buses** — things you could point at on a block diagram.
- The name says what happens: on each clock edge, values are **TRANSFERRED between REGISTERS**, possibly through some logic on the way.
- This is the level a Verilog designer actually works at, and it is the level the rest of Module 2 is about.

out/counter4_rtl.v — produced by Yosys in Lab 2

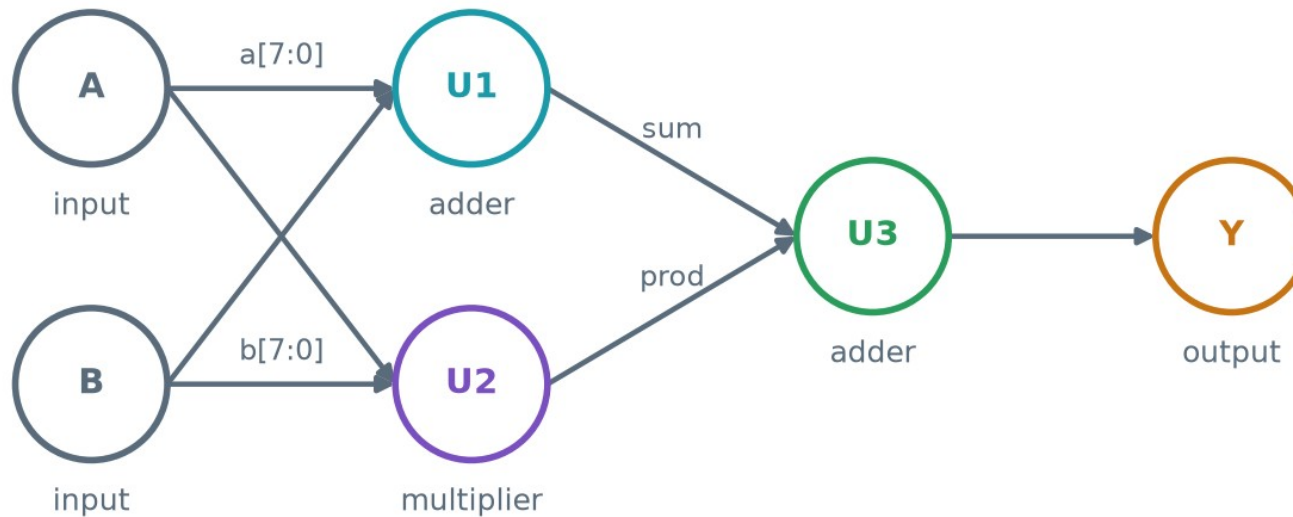
```
// the behavioural description above, as the tool re-wrote it:
assign _0_ = q + 4'h1;           // an incrementer
always @(posedge clk)           // a 4-bit register
    if (rst) q <= 4'h0;          // with a synchronous reset
    else    q <= _0_;
```

Two components: one adder and one register. That is what "data path design" means, and you will see Yosys report exactly those two cells.



A Netlist Is a Directed Graph

"The vertices indicate components, and the edges indicate interconnections."



VERTEX

one component

EDGE

a wire between two

THE WHOLE GRAPH

the design itself

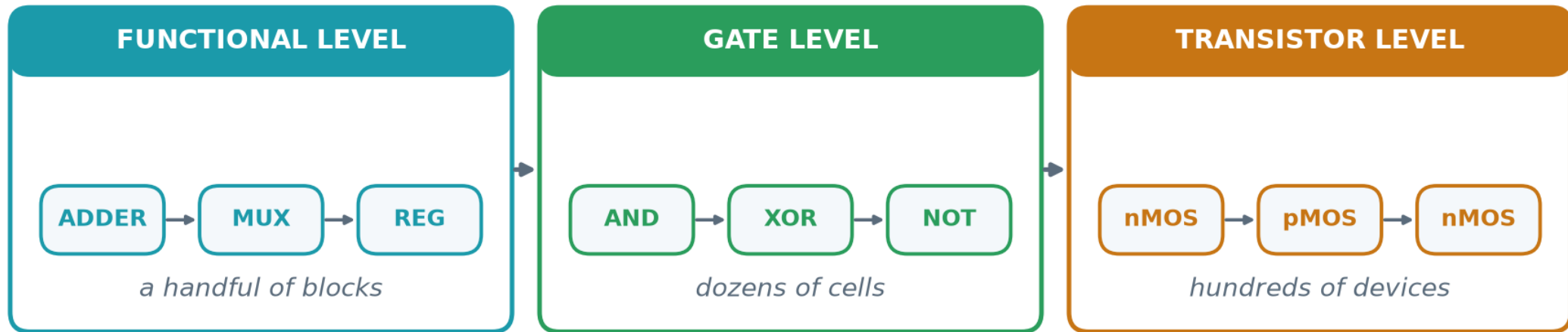
Why a graph, and not a drawing

Because a graph is something a program can hold in memory and transform. Every CAD step after this one — optimisation, mapping, placement, routing — is an operation on this graph.



The Same Netlist, at Three Levels

"Netlist may be specified at various levels, where the components may be functional modules, gates or transistors."



synthesis moves you left to right — never the other way

The same design, three times

Each level is a netlist. What changes is only what counts as a vertex. Synthesis is the process of rewriting a netlist in terms of smaller vertices.



Structural or Behavioural

BEHAVIOURAL — say WHAT it does

You describe the result. "The output is the sum of the two inputs." You say nothing about which gates are used, or how many. The synthesis tool chooses.

STRUCTURAL — say WHAT IT IS MADE OF

You name the parts and wire them together. "Instantiate a half adder here, another there, and an OR gate across their carries." This is a netlist, written as text.

Two consequences

A netlist IS a structural description — the lecturer says so explicitly. And synthesis is simply the act of turning one into the other: you write behavioural Verilog, and the tool emits structural Verilog.



4.3 · THE DISTINCTION

The Same Adder, Written Both Ways

```
// BEHAVIOURAL - say what it does
module add2 (input a, b, cin,
             output sum, cout);
    assign {cout, sum} = a + b + cin;
endmodule

// one line. The tool picks the gates.
```

```
// STRUCTURAL - say what it is made of
module add2 (input a, b, cin,
             output sum, cout);
    wire s1, c1, c2;
    xor u1 (s1, a, b);
    and u2 (c1, a, b);
    xor u3 (sum, s1, cin);
    and u4 (c2, s1, cin);
    or u5 (cout, c1, c2);
endmodule
```

Both are legal Verilog, and both describe the same adder

You will write far more behavioural code than structural. But the tool's OUTPUT is always structural — which is exactly why you have to be able to read it.



What a Standard Cell Is

"A pre-designed circuit module — like gates, flip-flops, a multiplexer — at the layout level."

THE SYMBOL



what you draw

THE BEHAVIOUR

a	b	y
0	0	1
0	1	1
1	0	1
1	1	0

what it computes

THE LAYOUT



the shapes on silicon

A cell is all of these at once

A standard cell is stored in a library as a symbol to place, a truth table to simulate, a timing model, and a finished piece of layout. Synthesis picks cells from that library; the place-and-route tool stamps their layouts down.



Why a Cell Library Exists

FEWER GATES

Fewer cells means less silicon area, and a cheaper chip. But the cheapest structure is often a deep one — and deep means slow.

FEWER GATE LEVELS

The depth of the logic is its delay, and delay is what limits your clock. Flattening it costs extra gates, and so costs area.

FEWER SIGNAL TRANSITIONS

Every time a gate output changes, charge moves and energy is spent. Reducing that switching is dynamic power saved — usually by adding control logic.

Why a library exists at all

Laying out a NAND gate correctly takes a skilled engineer a long time. Doing it once, perfectly, and then reusing it a million times is the only way the numbers work.



Three Goals That Pull Apart

GATES

Fewer cells, less silicon, cheaper chip — but the cheapest structure is often a deep one, and deep means slow.

LEVELS

The depth of the logic IS its delay, and delay is what limits the clock. Flattening it costs extra gates.

TRANSITIONS

Every change at a gate output moves charge and spends energy. Reducing switching usually means adding control logic.

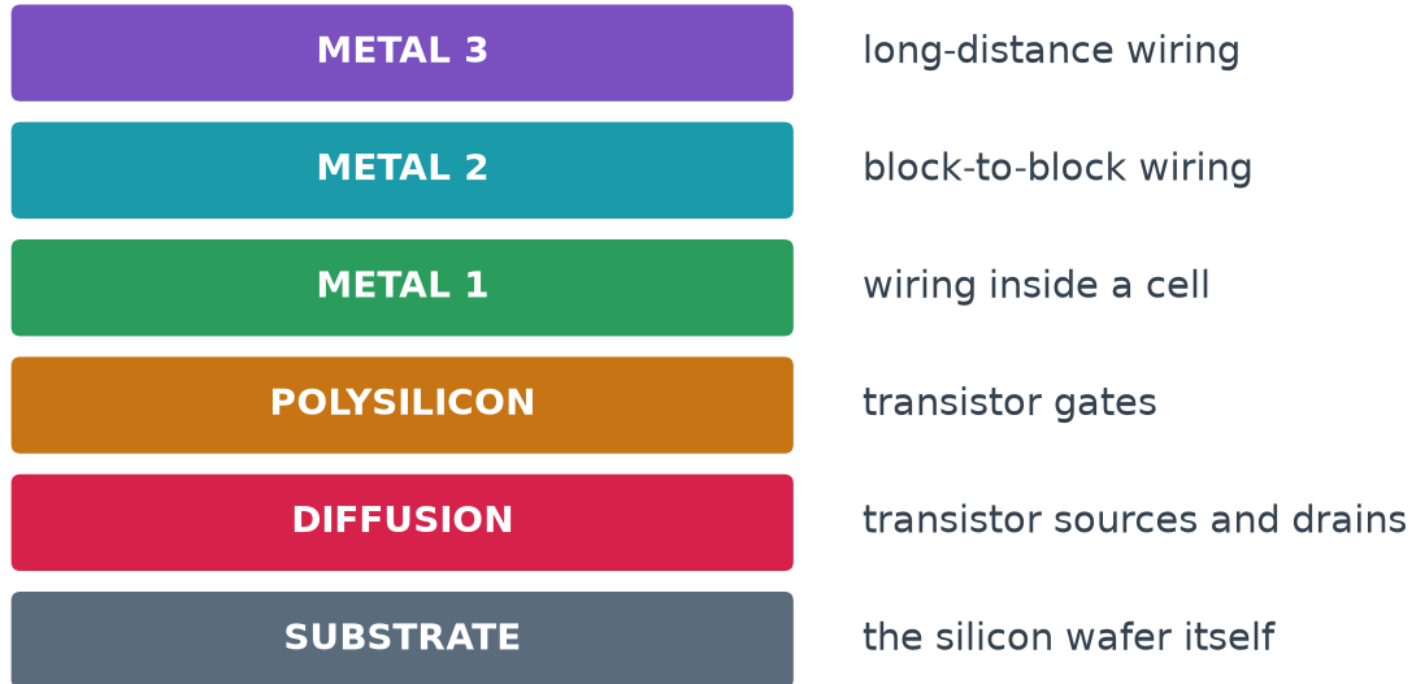
"These requirements are often conflicting"

A synthesis tool does not find THE answer — there isn't one. It finds an answer that satisfies the constraints you gave it. Give it different constraints and you get different hardware from the same Verilog.



Polygons, One Layer at a Time

a modern chip has 10–15 of these layers



Why the shapes are all rectangles

Because they are printed photographically, layer by layer, onto a silicon wafer. Regular shapes expose cleanly and etch predictably; awkward ones do not.



Two Ways to Finish: ASIC or FPGA

	ASIC (fabricate a chip)	FPGA (program a device)
What you receive	masks, then wafers from a foundry	a bitstream you load into a board
Turnaround	weeks to months	minutes
Cost to start	very high — mask sets dominate	the price of the board
Cost per unit at volume	very low	stays high
Speed	highest	lower — logic is configurable, not fixed
Can you change it after?	no — respin and pay again	yes, reprogram in the lab
Used for	high volume, or when speed decides	prototypes, small runs, teaching

"Much greater flexibility, but less speed"

The trade-off the lecturer names. An FPGA is a chip already manufactured, full of configurable blocks you program in your own laboratory. An ASIC is a chip built for your design alone.



Other Steps in the Design Flow

SIMULATION — to verify

Apply stimulus to the description and watch what comes out. Done at the logic, switch and circuit level. This is the one you will use constantly, from Lab 1 onwards.

FORMAL VERIFICATION

Prove mathematically that the design meets its specification, instead of testing it case by case. Powerful, and named here — but beyond this course.

TESTABILITY AND ATPG

Once a chip has been manufactured, you still have to check that THIS copy of it works. That needs patterns designed to expose manufacturing faults.

Where each of these returns in the course

Simulation is used in almost every lecture from Week 1 onwards, and Week 5 is devoted to test benches. Formal verification and testability are named here, but are beyond this course's scope.



Where the Rest of This Course Sits

Behavioral / Data Path

Weeks 1-4 · language features, description styles, procedural assignment, blocking vs non-blocking

Verification

Week 5 · test benches, and modelling finite state machines

Synthesis-aware coding

Week 6 · synthesizable Verilog, and recommended practices

Structure and case studies

Weeks 7-8 · memory, register banks, pipelining, switch level, and a complete processor

Notice what is missing

Nothing after logic design. Physical design, manufacturing and test are real and essential, but in this course they are the tools' job. Your work ends at a netlist.

PRACTICAL

Check It Yourself

Lecture 01 makes one big claim: a CAD tool rewrites a hardware description as a more detailed one without changing what it does. In this section you install four free tools and test that claim.

- Install Icarus Verilog, GTKWave, Yosys and GHDL — any platform
- Lab 1 · simulate a counter and look at its waveform
- Lab 2 · watch a tool rewrite your Verilog, and prove it still works
- Lab 3 · count the cells at each level of the flow
- Lab 4 · ASIC mapping against FPGA mapping
- Lab 5 · the same design in Verilog and in VHDL



The Toolchain for This Lecture

Four free programs, one for each thing the lecture says a CAD flow must do

ICARUS VERILOG the simulator

Compiles Verilog and runs it. This is how you check that a description behaves the way you meant. Command: iverilog, then vvp.

GTKWAVE the waveform viewer

Draws the signals a simulation recorded, so you can look at a clock edge and see what happened. Reads the .vcd file the test bench writes.

YOSYS the synthesiser

The CAD tool of the lecture's central slide: it reads your HDL and writes a more detailed HDL. This is what turns behaviour into gates.

GHDL the VHDL simulator

The same job as Icarus, for the other language — so you can compare the two on identical stimulus in Lab 5.

Why these four, and why they are free

Every tool here is open source and runs on Windows, macOS and Linux. Nothing in Lecture 01 needs a licence, a university server or a vendor account — which means you can do all of it on your own machine, tonight.



Installing It — One Line per Platform

LINUX (Ubuntu / Debian)

```
sudo apt update
sudo apt install -y \
    iverilog gtkwave yosys ghdl
```

macOS (Homebrew)

```
brew install icarus-verilog
brew install --cask gtkwave
brew install yosys ghdl
```

WINDOWS (via WSL)

```
wsl --install
# then, inside Ubuntu:
sudo apt install -y \
    iverilog gtkwave yosys ghdl
```

If you are on Windows, use WSL

Windows Subsystem for Linux gives you a real Ubuntu inside Windows, and every command in this lab then works unchanged. Install it once with `wsl --install` in an Administrator PowerShell, restart, and follow the Linux column.



Checking That It Worked

Run these four commands. Each must print a version number.

```
$ iverilog -V
```

→ Icarus Verilog version 12.0 (or later)

```
$ yosys -V
```

→ Yosys 0.33 (or later)

```
$ ghdl --version
```

→ GHDL 4.1.0 (or later)

```
$ gtkwave --version
```

→ GTKWave Analyzer v3.3 (or later)

If any of the four is missing, fix that before going on — every lab below needs all of them.

Versions newer than these are fine

The figures quoted throughout this deck were produced with Icarus 12.0, Yosys 0.33 and GHDL 4.1.0. A newer synthesiser may optimise slightly differently and report a different cell count — which is itself worth noticing, because it shows that optimisation has no single right answer.



The Five Labs, and What Each One Proves

LAB 1 make sim

Simulate the counter and look at its waveform. Proves you can run a description and see what it does — the "simulation for verification" step of the flow.

LAB 2 make transform make prove

Watch Yosys read your Verilog and write more Verilog, twice. Then run the SAME test bench against the gate netlist and get the same waveform. The lecture's central claim, checked.

LAB 3 make ladder

Count the cells at each level of the flow: 10 lines of source, then 2 RTL cells, then 10 gate-level cells. The ladder, in numbers.

LAB 4 make target

Map the same design for an ASIC and for an FPGA, and compare. Six gates and four flip-flops, against four LUTs and four flip-flops.

LAB 5 make langs

Write the counter again in VHDL, simulate both, and diff the waveforms. They match from the end of reset onwards — and where they differ, the difference is instructive.

Every claim in this lecture is checked by one of these

The theory section asserts that a CAD tool rewrites one hardware description as a more detailed one without changing what it does. Lab 2 proves exactly that, on your own machine, in about thirty seconds.



The Design: a 4-Bit Counter

rtl/counter4.v

```
// counter4 - a 4-bit up counter, written BEHAVIOURALLY.
//
// Nothing here says which gates to use, or how many. It says only
// what the output must be after each rising clock edge. Choosing
// the gates is the synthesis tool's job.
module counter4 (
    input wire      clk,
    input wire      rst,    // synchronous, active high
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (rst) q <= 4'd0;
        else     q <= q + 4'd1;
    end
endmodule
```

Ten lines of behaviour

This file never mentions a gate, a flip-flop or a wire between them. That is the whole point: it is a behavioural description, and everything below it in the flow will be produced from it by a program.



The Test Bench

rtl/tb_counter4.v (abridged)

```
`timescale 1ns/1ps
module tb_counter4;
    reg clk = 1'b0, rst = 1'b1;
    wire [3:0] q;
    counter4 dut (.clk(clk), .rst(rst), .q(q));

    always #5 clk = ~clk;          // 100 MHz: a 10 ns period

    initial begin
        $dumpfile("out/counter4.vcd"); // record for GTKWave
        $dumpvars(0, tb_counter4);
        @(negedge clk); rst = 1'b0;    // release reset
        ...                          // check 20 cycles
        $display("PASS counter4 counted 0..15 and wrapped");
    end
endmodule
```

A test bench is not hardware

It has no ports, it is never synthesised, and it may use delays, loops and \$display freely. It is a program whose job is to drive the hardware and watch what comes out.



HDL In, HDL Out — Twice

```
// 1. what you wrote
always @(posedge clk)
begin
    if (rst) q <= 4'd0;
    else    q <= q + 4'd1;
end
```

```
// 2. after synthesis (RTL)
assign _0_ = q + 4'h1;
always @(posedge clk)
    if (rst) q <= 4'h0;
    else    q <= _0_;
```

```
// 3. after mapping (gates)
assign _02_[0] = ~q[0];
assign _00_ = q[0] & q[1];
assign _01_ = ~(q[2] & _00_);
assign _03_[3] = ~(q[3] ^ _01_);
assign _03_[1] = q[0] ^ q[1];
assign _03_[2] = q[2] ^ _00_;
```

All three are Verilog

That is the lecture's central slide, made concrete. The tool did not compile your design into something else — it re-described it, twice, each time in more detail. The third column is a netlist: gates and the wires between them.



And It Still Behaves the Same

the whole claim of Lecture 01, in one command

```
$ make prove
```

```
--- 1. the behavioural description (what the designer wrote) ---  
PASS counter4 counted 0..15 and wrapped correctly
```

```
--- 2. synthesise it down to gates ---  
wrote out/counter4_gates.v (33 lines)
```

```
--- 3. the SAME test bench, run against the gate netlist ---  
PASS counter4 counted 0..15 and wrapped correctly
```

```
--- 4. compare the two waveforms, cycle by cycle ---  
IDENTICAL - 22 transitions, same value at the same time.  
The synthesiser changed the DESCRIPTION, not the BEHAVIOUR.
```

Why this is worth running yourself

One test bench, two very different descriptions, identical output at every instant. Nothing in the lecture has to be taken on trust after this.



What the Tools Actually Reported

Yosys 0.33, on rtl/counter4.v — reproduce it with `make ladder`

Level of the flow	Cells	What they are
1 · Behavioural source	—	10 lines of Verilog — no cells yet
2 · Data path (RTL)	2	1 adder, 1 four-bit register
3 · Logic, for an ASIC	10	4 flip-flops + AND, NAND, NOT, XNOR, XOR ×2
3 · Logic, for an FPGA	8	4 flip-flops + 4 look-up tables

Same circuit every time. Only the vocabulary changes — and the number of words needed to say it.

These are measured numbers, not illustrations

Every figure on this slide came out of Yosys 0.33 running on the counter in rtl/counter4.v. Run `make ladder` and you will get the same table. If your version of Yosys gives slightly different numbers, that is itself worth discussing — optimisation is not unique.



ASIC Mapping against FPGA Mapping

=== mapped to logic GATES (the ASIC route) ===

Number of cells:	10
\$_AND_	1
\$_NAND_	1
\$_NOT_	1
\$_SDFF_PP0_	4
\$_XNOR_	1
\$_XOR_	2

=== mapped to 4-input LUTs (the FPGA route) ===

Number of cells:	8
\$_SDFF_PP0_	4
\$lut	4

// four look-up tables replace
// the six gates on the left

Same source file, two different targets

The four flip-flops are the same either way — one per bit of the counter. What changes is the combinational logic: six dedicated gates for silicon you will manufacture, or four look-up tables inside a chip you can buy today. That is the flexibility-for-speed trade the lecturer names.



The Same Design in Two Languages

two languages, one circuit

```
$ make langs
--- Verilog (Icarus) ---      PASS   counted 0..15 and wrapped
--- VHDL (GHDL) ---          PASS   counted 0..15 and wrapped

--- the whole run, including time zero ---
they differ, and ONLY here:
    <    0.000 ns   xxxx          (Verilog)
    <    5.000 ns   0000
    >    0.000 ns   0000          (VHDL)

--- from the end of reset onwards (15 ns) ---
IDENTICAL - 20 transitions, same value at the same time
```

The one difference is the most useful part of the lab

A Verilog reg holds x — unknown — until something drives it, so q reads xxxx until the reset edge clears it. The VHDL signal was declared with an initial value, so it reads 0000 from time zero. The hardware is identical: a real flip-flop also powers up unpredictably, which is why designs have resets.



Take It Further

- 1. Change the counter to 8 bits. Re-run `make_ladder` and record the new cell counts. Did the number of gates double?
- 2. Make the reset asynchronous — always `@(posedge clk or posedge rst)` — and compare the gate-level netlists before and after.
- 3. Write a down-counter and prove it with `make_prove`. How many cells does it take compared with counting up?
- 4. Open both `out/counter4_behav.vcd` and `out/counter4_gates.vcd` in GTKWave at the same time and align them.
- 5. Try `abc -lut 6` instead of `abc -lut 4`. Why do larger look-up tables need fewer of them?
- 6. Write the counter in VHDL from memory, then `diff` your version against `vhdl/counter4.vhd`.

The workbook carries all of these with full worked solutions, plus thirty more.



CHECKPOINT

Ten Questions on Lecture 01

1. Name the five levels of the design flow, in order.
2. What are the INPUT and the OUTPUT of a CAD tool?
3. Define a netlist.
4. What does "register transfer" refer to?
5. Give two ways of specifying behaviour.

6. What is a standard cell, and why does a library of them exist?
7. State Moore's Law. Is it a law of physics?
8. Name three optimisation goals that conflict.
9. Give one advantage of an FPGA over an ASIC, and one the other way.
10. Why is a netlist also called a structural description?

Answers

Every one of these is answered somewhere in this deck, and all ten appear with model answers in the workbook. Questions 2, 3 and 10 are the ones that matter most for the rest of the module.



What Lecture 01 Established

THE PROBLEM

Chips grew from a handful of transistors to billions. No human can place them, and the requirements — area, speed, power — conflict.

THE ANSWER

A standardized design flow: specification, behavioural design, data path, logic, physical, manufacturing. Each step has a defined input and output, and each is carried out by a program.

THE MECHANISM

A CAD tool reads a hardware description and writes a more detailed hardware description. It never changes what the circuit does — you proved this yourself in Lab 2.

THE LANGUAGE

Those descriptions are written in an HDL. Verilog and VHDL are the two in common use; this course uses Verilog. An HDL is not a programming language.

YOUR PLACE IN IT

A Verilog designer works at the behavioural and data path levels. Everything below is the tools' job — which is why those two levels are what the next seven weeks teach.



NEXT

Lecture 02 — Design Representation

The next lecture takes the one idea this one left implicit — that the same design can be looked at from different angles — and makes it the subject.

BEHAVIOURAL

Algorithms, specifications, Boolean expressions, truth tables, state diagrams.

STRUCTURAL

Netlists: modules, gates and transistors, and the wires between them.

PHYSICAL

Floorplans, cell placement, routing, and the final layout geometry.

The Y-diagram

Lecture 02 arranges those three as the arms of a Y, with the highest level of abstraction at the outside and the finished layout at the centre. Every design step is then a move along an arm, or a jump between arms — and Module 2 lives on the behavioural and structural arms.